

## ДОМАШНЕЕ ЗАДАНИЕ № 6.

### Решение задач многоэкстремальной оптимизации

#### Методические указания.

#### 1. Случайный мултистарт

##### Цель работы:

1. Изучение методов решения задач многоэкстремальной оптимизации.
2. Разработка программ реализации алгоритмов мултистарта (метода ближайшего соседа, метода конкурирующих точек).
3. Решение задачи многоэкстремальной оптимизации для функции Шекеля.

##### Постановка задачи

Найти минимумы функции Шекеля (глобальный минимум, в частном случае):

$$f(x) = -\sum_{i=1}^3 \frac{a}{f_i^0 + b \sum_{j=1}^3 (x_j - x_{ij}^0)^2}, \quad \sum_{i=1}^3 \frac{1}{f_i^0} = f^0, \quad (1)$$
$$f^0 = -c, \quad x_{ij}^0 \in D \subset [\alpha, \beta]^3, \quad i, j = 1, 2, 3.$$

##### Стратегия поиска

Стратегия метода случайного мултистарта состоит в многократном решении задачи локального поиска экстремума из случайных начальных точек. Случайный мултистарт является наиболее известным и широко используемым представителем семейства методов мултистарта. В чистом виде случайный мултистарт является неэффективным алгоритмом, поскольку при его использовании большое количество ненужной вычислительной работы затрачивается на повторные спуски к точкам локальных минимумов. Однако его модификации, называемые методами конкурирующих точек и основанные на использовании процедур кластеризации для предотвращения повторных спусков, являются одними из наиболее эффективных алгоритмов.

Основная сложность при практической реализации метода мултистарта состоит в следующем. Для того чтобы с высокой надежностью отыскать точку глобального минимума, необходимо начальных точек для локальных алгоритмов взять существенно больше, чем число локальных минимумов функции  $f(x)$ , (которое к тому же обычно неизвестно). Но при этом большая часть вычислительных затрат будет связана с нахождением точек локальных минимумов, обнаруженных ранее. Для преодоления этой сложности мултистарт обычно модифицируют одним из следующих способов, основанных на эвристических предположениях о том, что точки локальных минимумов находятся достаточно далеко друг от друга.

*Первый способ* состоит в том, что найденные точки локальных минимумов окружаются некоторыми окрестностями и попадание других точек в эти окрестности считаются равносильными попаданию в точки локальных минимумов. Корректное использование получающегося метода возможно лишь в тех случаях, когда можно точно указать размер и форму соответствующих окрестностей, либо целиком лежащих внутри области притяжения точек локальных минимумов, либо гарантированно не содержащих точек локального минимума.

*Второй способ*, называемый иногда методом *конкурирующих точек*, состоит в том, что поиск локальных экстремумов производится одновременно (параллельно), причем точки, расстояние между которыми не превышает некоторого фиксированного, достаточно малого числа, объединяются, т.е. заменяются одной из них – той, значение функции в которой меньше. В качестве начальных точек для локального поиска обычно выбирают точки какой-либо из равномерных на  $X$  сеток. Наибольшей популярностью пользуются случайные сетки и  $\Pi\tau$ -сетки. Число начальных точек должно существенно превышать ожидаемое число локальных минимумов целевой функции. Проблема объединения точек является проблемой *кластерного анализа*, и поэтому любой метод кластерного анализа может быть использован в указанных целях. Одним из наиболее простых и популярных является метод *ближайшего соседа*, основанный на предположении, что расстояние  $\rho_{ij} = \rho(z_i, z_j)$  в некоторой метрике  $\rho$  между точками  $z_i$  и  $z_j$ , принадлежащими окрестности одного локального минимума, меньше расстояний между точками, принадлежащими окрестностям разных минимумов.

### Алгоритм метода ближайшего соседа

**Ш.1.** Генерируются  $K$  точек  $(z_1, z_2, \dots, z_K)$  случайным образом с помощью, например, алгоритма Монте-Карло. Считается, что все они принадлежат различным кластерам (кластер ассоциируется с окрестностью локального минимума).

**Ш.2.** Находится ближайшая пара точек  $z_i, z_j$ , т.е. такая пара, для которой  $\rho_{ij} = \arg \min_{k,l=1,\dots,K} \rho_{kl}$ .

**Ш.3.** Если расстояние  $\rho_{ij} = \rho(z_i, z_j)$  между ближайшими соседями не превосходит некоторое достаточно малое число  $\delta > 0$ , то точки  $z_i$  и  $z_j$ , а также соответствующие им кластеры объединяются и число кластеров  $K$  уменьшается на единицу. После этого происходит переход на Ш.2.

**Ш.4.** Остановка алгоритма происходит, либо если расстояние между ближайшими соседями превосходит  $\delta$  либо если остается лишь один кластер.

В качестве метрики  $\rho(z_i, z_j)$  обычно выбирается евклидова метрика. Качество алгоритма как процедуры кластеризации существенно зависит от того, насколько удачно выбрано число  $\delta$ . Это число должно быть достаточно малым, во всяком случае меньше расстояний между соседними точками локальных минимумов.

### Алгоритм метода конкурирующих точек

**Ш.1.** Моделируется равномерное на  $X$  распределение. В результате получается  $N$  точек  $\{x_1, x_2, \dots, x_N\}$ .

**Ш.2.** Используя точки  $\{x_1, x_2, \dots, x_N\}$  в качестве начальных, проводится одна или несколько итераций какого-либо алгоритма локальной оптимизации. Получаем точки  $z_1, \dots, z_N$ .

**Ш.3.** Применяется какой-либо метод кластеризации к точкам  $z_1, \dots, z_N$ . Пусть  $m$  - число получившихся кластеров. Если  $m = 1$ , то переход на Ш.5. Иначе переход на Ш.4.

**Ш.4.** Выбираются представители  $x_1, x_2, \dots, x_m$  от всех кластеров (естественно выбирать в кластерах с наименьшим значением целевой функции). Положим  $N = m$  и переход на Ш.2.

**Ш.5.** Считаем, что находимся в окрестности точки глобального минимума. Выбрать представителя от единственного кластера. Используя его в качестве начальной точки для алгоритма локальной оптимизации, обладающего высокой скоростью сходимости в окрестности точки экстремума.

### Задание

Требуется найти глобальный минимум функции Шекеля.

### Варианты задания:

$$1. \alpha = 0, \beta = 5. \begin{cases} a = 2, \\ b = 3, \\ c = 1. \end{cases}$$

$$2. \alpha = 2, \beta = 8. \begin{cases} a = 1, \\ b = 3, \\ c = 2. \end{cases}$$

$$3. \alpha = 3, \beta = 11. \begin{cases} a = 1, \\ b = 4, \\ c = 2. \end{cases}$$

$$4. \alpha = 1, \beta = 7. \begin{cases} a = 3, \\ b = 3, \\ c = 3. \end{cases}$$

$$5. \alpha = 1, \beta = 6. \begin{cases} a = 2, \\ b = 2, \\ c = 1. \end{cases}$$

$$6. \alpha = 0, \beta = 8. \begin{cases} a = 1, \\ b = 1, \\ c = 2. \end{cases}$$

$$7. \alpha = 2, \beta = 9. \begin{cases} a = 1, \\ b = 3, \\ c = 2. \end{cases}$$

$$8. \alpha = 0, \beta = 5. \begin{cases} a = 2, \\ b = 3, \\ c = 1. \end{cases}$$

1. Реализовать алгоритмы многоэкстремальной оптимизации методом мултистарта на одном из языков высокого уровня (C++, C#, Python, Haskell и др.).
2. Решить задачу многоэкстремальной оптимизации с помощью метода мултистарта.

### 2. Генетический алгоритм.

#### Цель работы:

1. Изучение методов решения задач многоэкстремальной оптимизации на основе генетического алгоритма.
2. Разработка программы реализации генетического алгоритма.
3. Решение задачи многоэкстремальной оптимизации для заданных многоэкстремальных функций.

## Постановка задачи

Дана целевая функция  $f(x) = f(x_1, x_2, \dots, x_n)$ , определенная на множестве допустимых решений  $D \subseteq R^n$ . Требуется найти глобальные минимумы заданных функций  $f(x)$  на допустимом множестве  $D$ . То есть такую точку

$$x^e = \operatorname{Arg} \min_{x \in D} f(x), \text{ где } x = (x_1, \dots, x_n)^T, D = \{x | x_i \in [\alpha, \beta], i = 1, \dots, n\}. \quad (2)$$

Задача поиска максимума целевой функции  $f(X)$  сводится к задаче поиска минимума путем замены знака перед функцией на противоположный:

$$f(x^e) = \max_{x \in D} f(x) = -\min_{x \in D} [-f(x)].$$

## Стратегия поиска

Генетические алгоритмы имитируют природные способы оптимизации, присущие процессам эволюции живых систем. А именно:

- генетическое наследование;
- изменчивость;
- естественный отбор.

**Целевая функция**  $f(x)$  соответствует природному понятию *приспособленности* живого организма. Вектор переменных  $x = (x_1, \dots, x_n)^T$  целевой функции называется **фенотипом**, а отдельные его параметры – **признаками**  $i = 1, 2, \dots, n$ .

Любой живой организм может быть представлен своим генотипом и фенотипом.

**Генотип** – это совокупность наследственных признаков, информация о которых заключена в хромосомном наборе генов.

**Фенотип** – совокупность всех признаков и свойств организма, формирующихся в процессе взаимодействия его генотипа и внешней среды. Каждый ген имеет своё отражение в фенотипе.

**Генетические алгоритмы** ведут поиск решения на уровне генотипа. Каждую координату  $x_i$  вектора  $x = (x_1, \dots, x_n)^T \in D$  представляют в некоторой форме  $s_i$ , удобной для использования в генетическом алгоритме и называется *геном*. Для этого необходимо выполнить преобразование, в общем случае не взаимно однозначное, вектора переменных  $x = (x_1, \dots, x_n)^T \in D$  в некоторую структуру  $s = (s_1, s_2, \dots, s_n)^T \in S$ , называемую **хромосомой**

(**генотипом, особью**):  $D \xrightarrow{e} S$ , где  $e$  функция кодирования,  $S$  - пространство представлений (как правило  $D \neq S$ ).

Для того, чтобы восстанавливать по хромосоме решение, необходимо задать обратное преобразование  $S \xrightarrow{e^{-1}} D$ , где  $e^{-1}$  - функция декодирования.

В пространстве представлений  $S$  вводится так называемая **функция приспособленности (функция фитнеса)**  $\mu(s): S \xrightarrow{\mu} R$ , где  $R$  - множество вещественных чисел, аналогичная целевой функции  $f(x)$  на множестве  $D$ . Функция  $\mu(s)$  может быть любая функция, удовлетворяющая условию:

$$\forall x^1, x^2 \in D: s^1 = e(x^1), s^2 = e(x^2), s^1 \neq s^2, \text{ если } f(x^1) > f(x^2), \text{ то } \mu(s^1) > \mu(s^2).$$

При решении используются конечные наборы:

$$I = \{s^k = (s_1^k, s_2^k, \dots, s_n^k)^T, k = 1, 2, \dots, m\} \subset S$$

Возможных решений, называемых **популяциями**, где  $s^k$  - хромосомы с номером  $k$ ,  $m$  - размер популяции,  $s_i^k$  - ген с номером  $i$   $k$ -той популяции.

Затем осуществляется обратное преобразование:

$$x^e = e^{-1}(s^*).$$

Различают два способа кодирования:

1. Бинарное кодирование.
2. Вещественное кодирование.

Будем использовать второй вариант кодирования. В этом случае целевая функция может использоваться непосредственно как функция фитнеса. Тогда в качестве функции фитнеса  $\mu(x)$  получается как преобразование целевой функции, т.е. функция фитнеса  $\mu(x): D \xrightarrow{\mu} R$ , где  $R$  - множество вещественных чисел, аналогична целевой функции  $f(x)$ .

Функцией  $\mu(x)$  может быть любая функция, удовлетворяющая следующему условию:

$$\forall x^1, x^2 \in D: x^1 \neq x^2, \text{ если } f(x^1) > f(x^2), \text{ то } \mu(x^1) > \mu(x^2).$$

Решение исходной оптимизационной задачи  $f(x^*) = \min_{x \in D} f(x)$  сводится к поиску решения  $x_\mu^*$  другой оптимизационной задачи:

$$\mu(x_\mu^*) = \max_{x \in D} \mu(x). \quad (3)$$

В силу выбора функции  $\mu(x)$ , решение задач (1) и (2) (хромосома) совпадают:

$$x^e = x^* = \text{Arg min}_{x \in D} f(x) = \text{Arg max}_{x \in D} \mu(x) = x_\mu^*. \quad (4)$$

При решении задачи (2) используются конечные наборы  $I = \{x^k = (x_1^k, \dots, x_n^k)^T, k = 1, 2, \dots, Mp\} \subset D$  возможных решений, называемых популяциями, где  $x^k$  - хромосома с номером  $k$ ,  $M$  - размер популяции,  $x_i^k$  - ген с номером  $i$ .

## Генетический алгоритм.

### III.1. Формирование исходной популяции.

III.1.1. Задается номер популяции  $t = 0$ , максимальное количество популяций  $Np$ , номер итерации цикла  $k = 1$ , размер популяции  $Mp$ .

III.1.2. Случайным образом выбирается начальная точка  $x^0$  - исходная хромосома. Она может быть выбрана как внутренняя точка гиперкуба области допустимых значений  $D$ . Из этой точки формируется исходная популяция. Для этого с помощью равномерного распределения на единичном отрезке  $[0,1]$   $Mp$  раз генерируется последовательность из  $n$  случайных точек  $\{P_i^{0k}\}_{i=1,n}^{k=1,Mp}$ ,  $i = 1, \dots, n$ ;  $k = 1, \dots, Mp$ . Используя линейное преобразование, каждая точка отображается на соответствующий ей промежуток  $[\alpha, \beta]$ :  $P_i^k = (\beta_i - \alpha_i)P_i^{0k} + \alpha_i$ . Составляя векторы из точек последовательности  $\{P_i^k\}$  при фиксированных  $k$ , получаем  $Mp$  начальных векторов  $x^k = (x_1^k, \dots, x_n^k)^T$ ,  $x_i^k = P_i^k$ ,  $i = 1, \dots, n$ , координаты которых  $x_i$  имеют равномерное распределение на отрезках  $[\alpha_i, \beta_i]$ ,  $i = 1, \dots, n$ . Таким образом, может быть сформирована начальная популяция  $I_0 = \{x^k, k = 1, \dots, Mp \mid x^k = (x_1^k, x_2^k, \dots, x_n^k) \in D\}$ .

III.1.3. Вычисляется значение функции фитнеса для каждой особи

$$x^k \in I_0: \mu_k = \mu(x^k), k = 1, \dots, Mp \text{ и популяции } I_0 \text{ в целом } \mu = \sum_{k=1}^{Mp} \mu_i.$$

### III.2. Отбор (селекция).

Селекция – это операция, которая осуществляет отбор особей (хромосом)  $x^k$  в соответствии со значениями функции фитнеса  $\mu(x^k)$  для последующего их скрещивания.

Ш.2.1. Вычислить кумулятивную вероятность  $q_i = \sum_{j=1}^i \mu_j(x^j)$ ,  $i = 1, 2, \dots, Mp$ .

Ш.2.2. Сформировать случайное действительное число  $r$  в интервале  $(0, Mp]$ .

Ш.2.3. Выбрать  $i$ -ю хромосому  $x^i$  ( $1 \leq i \leq Mp$ ) так, чтобы  $q_{i-1} < r \leq q_i$ .

Ш.2.4. Перейти на Ш.2.2. до тех пор, пока не будет сформирована новая популяция ( $while(i \leq Mp)$ ).

### Ш.3. Кроссинговер (скрещивание).

Скрещивание – это операция, при которой из нескольких, обычно двух хромосом (особей), называемых родителями, порождается одна или несколько новых, называемых потомками.

Ш.3.1. Определяется параметр  $Pc \in (0, 1]$  как вероятность кроссинговера. Эта вероятность дает ожидаемое число  $Pc \cdot Mp$  хромосом, подвергаемых операции кроссинговера.

Ш.3.2. Для операции кроссинговера выполняется процесс, повторяющийся от  $i = 1$  до  $Pc \cdot Mp$ : формируется случайное действительное число  $r$  из сегмента  $[0, 1]$ , при этом, если  $r < Pc$ , то хромосома  $x^i$  выбирается как родительская.

Ш.3.3. Отбираются пары родительских хромосом  $(x^i, x^j)$ , где  $i \neq j$ . Действие оператора кроссинговера осуществляется следующим образом:

Ш.3.4. Формируется случайное число  $c \in (0, 1)$ , затем оператор кроссинговера, действующий на исходные пары  $(x^i, x^j)$ , производит две хромосомы потомки  $X$  и  $Y$ :

$$X = c \cdot x^i + (1 - c) \cdot x^j, \quad Y = (1 - c) \cdot x^i + c \cdot x^j.$$

Ш.3.5. Если допустимое множество является выпуклым, то кроссинговер обеспечивает допустимость обоих потомков, в случае если допустимы оба родителя. Следует проверить допустимость каждого потомка перед тем, как он будет включен в новую популяцию. Если оба потомка являются допустимыми, тогда родители заменяются этими потомками. Если это не так, сохраняется допустимый потомок, если он существует, а затем вновь выполняется оператор кроссинговера с новым значением случайного числа  $c$  до тех пор, пока не будут получены два новых допустимых потомка или не будет превышено заданное число циклов. В этом случае осуществляется замена родителей только теми (сохраненными ранее) потомками, которые оказались допустимыми.

### Ш.4. Мутация.

Мутация – это преобразование хромосомы, случайно изменяющее один или несколько из её генов. Оператор мутации предназначен для того, чтобы поддерживать разнообразие особей в популяции.

Ш.4.1. Определим параметр  $Pm \in (0, 1]$  как вероятность мутации. Эта вероятность дает ожидаемое число  $Pm \cdot Mp$  хромосом, подвергаемых операции мутации.

Ш.4.2. Для операции мутации выполняется процесс, повторяющийся от  $i = 1$  до  $Pm \cdot Mp$ : формируется случайное действительное число  $r$  из сегмента  $[0, 1]$ , при этом, если  $r < Pm$ , то хромосома  $x^i$  выбирается как родительская для операции мутации. Для каждой выбранной родительской хромосомы  $x^i$ , обозначенной как  $Z = (x_1, x_2, \dots, x_n)$ , производится мутация.

Ш.4.3. Поочередно рассматривается каждый потомок из ожидаемого числа  $Pm \cdot Mp$  хромосом. Среди генов выбранной родительской хромосомы  $Z = (x_1, x_2, \dots, x_n)$  случайно (с вероятностью  $1/n$ ) выбирается один с номером  $p \in (1, 2, \dots, n)$  подлежащий замене. Его новое

значение  $x_p^M$  случайным образом выбирается из промежутка  $[\alpha_p, \beta_p]$  изменения выбранной координаты  $x_p$ .

### **Ш.5. Формирование новой популяции.**

Ш.5.1. С равной вероятностью из потомков мутантов предыдущего шага выбирается один  $x^M = (x_1, x_2, \dots, x_p^M, \dots, x_n)$ .

Ш.5.2. Выбранный потомок добавляется в популяцию вместо хромосомы, которой соответствует наименьшее значение функции фитнеса (наихудшее из допустимых значений).

Ш.5.3 Вычисляется значение функции фитнеса для мутантного потомка  $\mu_M = \mu(x^M)$ .

Ш.5.4. Проверка условий:

Ш.5.4.1. Если  $k < Mp$ , то  $k = k + 1$  и переход на Ш.2.

Ш.5.4.2. Если  $k = Mp$ , то  $t = t + 1$  и переход на Ш.6.

### **Ш.6. Проверка условия останова генетического алгоритма.**

Условием окончания работы генетического алгоритма является формирование заданного количества популяций  $t = Nr$ .

Ш.6.1. Если условие не выполнено, то полагаем  $k = 1$  и переход на Ш.2.

Ш.6.2. Если условие окончания работы выполнено, то в качестве решения (приближенного) задачи  $\mu(x_\mu^*) = \max_{x \in D} \mu(x)$  выбирается особь с лучшим значением функции фитнеса из текущей популяции:  $x_\mu^* \cong x_\mu^e = \text{Arg max } \mu(x^k)$ , а по нему определяется приближенное решение поставленной задачи  $f(x^*) = \min f(x) : x^* = x_\mu^*$ .

**Замечание 1.** В качестве функции фитнеса можно использовать обратную целевую функцию  $\mu(x) = \frac{1}{f(x)}$ .

**Замечание 2.** Обычно размер популяции выбирают в пределах 30-60 особей.

**Замечание 3.** Вероятность кроссинговера принимается равной  $Pc = 0.3 - 0.5$ , вероятность мутации  $Pm = 0.05 - 0.2$ .

### Варианты задания:

1.  $f_1(x) = -\sum_{i=1}^{30} x_i \sin(\sqrt{|x_i|}), n = 30, \alpha = -2, \beta = 4;$
2.  $f_2(x) = \left( \frac{1}{200} + \sum_{j=1}^{10} \frac{1}{j + \sum_{i=1}^2 (x_i - a_{ij})^4} \right)^{-1}, n = 2, a_{ij} = i + j, \alpha = 0, \beta = 5;$
3.  $f_3(x) = \sum_{i=1}^{11} \left[ a_i - \frac{x_1(b_i^2 + b_i x_2)}{b_i^2 + b_i x_3 + x_4} \right]^2, n = 4, a_i = 2i, b_i = a_i^n, \alpha = -1, \beta = 10;$
4.  $f_4(x) = 4x_1 + 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 3x_2^4, n = 2, \alpha = 2, \beta = 15;$
5.  $f_5(x) = \left( x_2 - \frac{5.1}{4\pi^2} x_1^2 + \frac{5}{\pi} x_1 - 6 \right) + 10 \left( 1 - \frac{1}{8\pi} \right) \cos x_1 + 10, n = 2, \alpha = 1, \beta = 14;$
6.  $f_6(x) = \sum_{i=1}^n [x_i^2 - 10 \cos(2\pi x_i) + 10], n = 3, \alpha = 0, \beta = 20;$
7.  $f_7(x) = -20 \exp \left( -0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left( \frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i) \right) + 20, n = 3, \alpha = 0.5, \beta = 12;$
8.  $f_8(x) = \frac{1}{200} \sum_{i=1}^n x_i^2 - \prod_{i=1}^n \cos \left( \frac{x_i}{\sqrt{i}} \right) + 1, n = 4, \alpha = 0.8, \beta = 20;$

1. Реализовать генетический алгоритм многоэкстремальной оптимизации на одном из языков высокого уровня (C++, C#, Python, Haskell и др.).
2. Решить задачу многоэкстремальной оптимизации с помощью генетического алгоритма.

#### Требования к отчету.

1. Отчет должен содержать:
  - 1.1. титульный лист;
  - 1.2. цель работы;
  - 1.3. постановку задачи;
  - 1.4. проверку решения на допустимость.
2. Исследование выполнить с помощью написанной Вами программы с результатами в виде таблицы.
3. Кроме текста исследования следует привести также текст исходного кода программ.
4. Отчет оформляется в формате PDF желательно в редакторе TEX.